



Pashov Audit Group

Paxos Labs Security Review

September 19th 2026 - September 20th 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. Methodology	4
5. Executive Summary	4
6. Findings	4
Info Severity	5
[I-01] Hook documentation overstates protection scope	5
[I-02] Shipped Unit Test Asserts the Wrong Custom Error, Failing the Suite and Leaving the Reentrancy Branch Unasserted	6
[I-03] Probe burns full 30k stipend on every transfer	7
[I-04] Hook installation leaves existing share-offer requests appearing solvable	8
[I-05] Queue-wide reentrancy lock blocks unrelated BoringVault transfers	10
[Self-report I-01] Caller can underfund reentrancy probe gas stipend, bypassing it	11



1. About Pashov Audit Group

Pashov Audit Group consists of 100+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 500 security audits completed - uncovering and helping patch thousands of vulnerabilities - the group strives to create the most robust and pleasant audit experience possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Telegram [@pashovkrum](#).

2. Disclaimer

A security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where the researchers try to find as many issues as possible. We can not guarantee 100% security after the review. Subsequent reviews, bug bounty programs and on-chain monitoring are strongly recommended. The client's remediation of each issue has been reviewed; the outcome is recorded in the finding's Resolution section.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Critical

Loss or compromise of critical assets, or protocol-wide impact.

High

Significant material loss of assets, or serious harm to a group of users.

Medium

Moderate material loss, or moderate harm, often under conditional assumptions.

Low

Minor impact, limited scope, or requires unlikely conditions to trigger.

Info

No direct security impact - best-practice, code-quality or hardening suggestion.

IMPACT

High

- leads to a significant material loss of assets in the protocol or significantly harms a group of users

Medium

- leads to a moderate material loss of assets in the protocol or moderately harms a group of users

Low

- leads to a minor material loss of assets in the protocol or harms a small group of users

LIKELIHOOD

High

- attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost

Medium

- only a conditionally incentivized attack vector, but still relatively likely



Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. Methodology

A large team of security researchers hunt vulnerabilities on the scope provided during a time-boxed review.

Every issue submission is deduplicated and validated by a judging team. Every accepted issue appears below with its impact and a recommended remediation. The client's response to each issue was then reviewed, and its resolution status - fixed, acknowledged, or invalidated - is recorded with the finding.

5. Executive Summary

A time-boxed security review of Paxos Labs was conducted by the Pashov Audit Group team. This report documents 5 issues across the reviewed codebase, reported by Auditor-Nate, BRDNS, InfiniteSec, IvanFitro, JeRRy, K42, Slayer Security, Tejas.

SCOPE

<https://github.com/paxoslabs/pxl-boring-vault/pull/361>

FIXES REVIEW COMMIT HASH

[75bda1c32dc6fb77ae792426755d28477ed1626f](https://github.com/paxoslabs/pxl-boring-vault/pull/361)

6. Findings

ID	TITLE	SEVERITY	STATUS
I-01	Hook documentation overstates protection scope	Info	Fixed
I-02	Shipped Unit Test Asserts the Wrong Custom Error, Failing the Suite and Leaving the Reentrancy Branch Unasserted	Info	Fixed
I-03	Probe burns full 30k stipend on every transfer	Info	Fixed
I-04	Hook installation leaves existing share-offer requests appearing solvable	Info	Acknowledged
I-05	Queue-wide reentrancy lock blocks unrelated BoringVault transfers	Info	Acknowledged
Self-report I-01	Caller can underfund reentrancy probe gas stipend, bypassing it	Info	Fixed

Info Severity

[I-01] Hook documentation overstates protection scope

Info

Fixed

SUMMARY

The NatSpec on `AtomicQueueDerailBeforeTransferHook` presents the hook as a complete fix, stating "To make this attack impossible on old vaults that used this contract" (lines 11-12). The actual protection is narrower on three axes, none of which is documented:

- Only share transfers of a vault that installs this hook are checked. Any other ERC20 a user approved to AtomicQueue stays drainable through the same solver-substitution attack, because a vault hook cannot observe those transfers. The fix for those approvals is revocation, not this contract.
- Only one configured queue is probed (`atomicQueue`, line 26). The hook never learns which spender initiated a transfer, so approvals to any other AtomicQueue instance stay outside its view.
- Protection starts only after the owner calls `setBeforeTransferHook`, and that call replaces whatever hook a vault already held.

The contract is an emergency mitigation whose primary consumer is an operator deciding whether users are safe. Its documentation accuracy carries security weight because the contract offers no runtime signal for any of these gaps.

IMPACT

Text that reads as "attack impossible" invites operators to skip approval revocation, multi-queue coverage, and review of any prior hook, leaving users exposed on the uncovered paths.

The header also retains spelling errors a review commit was meant to remove: `reenterency` at lines 13, 14, 39, and 43, and `reenternacy` at line 35. Line 42 refers to a succeeded transaction where a succeeded call is meant; `beforeTransfer` never observes transaction outcome, only its own `staticcall`.

RECOMMENDATION

Rewrite the header NatSpec to state the exact scope:

```
/**
 * @dev Blocks BoringVault share transfers that occur while `atomicQueue.solve` is executing,
 * by probing that queue's reentrancy guard on every transfer.
 * Scope limits:
 * - Protects only share transfers of vaults that install this hook.
 * - Probes only `atomicQueue`; approvals to any other queue are not covered.
 * - Does not protect other ERC20 approvals granted to AtomicQueue; users must revoke them.
 * - Installing this hook replaces any hook a vault already holds.
 */
```

Fix the spelling to `reentrancy` at lines 13, 14, 35, 39, and 43, and change "transaction succeeded" to "call succeeded" at line 42. Add a deployment checklist entry that lists approval revocation and per-queue coverage as separate steps.



RESOLUTION

Paxos Labs

Fixed. Documentation coverage has been improved and spelling errors have been fixed.

[I-02] Shipped Unit Test Asserts the Wrong Custom Error, Failing the Suite and Leaving the Reentrancy Branch Unasserted

Info

Fixed

SUMMARY

`AtomicQueueDerailBeforeTransferHook` classifies a non-empty probe revert into two custom errors. It returns `UseOfInvalidContract` only when the return data exactly matches the solmate reentrancy string, and `UnexpectedRevert` for everything else.

```
bytes32 internal constant REENTRANCY_REVERT_HASH =  
    keccak256(abi.encodeWithSignature("Error(string)", "REENTRANCY"));
```

```
if (returnData.length == 0) return;  
if (keccak256(returnData) == REENTRANCY_REVERT_HASH) {  
    revert UseOfInvalidContract(from, address(atomicQueue), returnData);  
}  
revert UnexpectedRevert(from, returnData);
```

The test `testUseOfInvalidContractBlocksTransfers` points the hook at `RevertingQueue`, whose `solve` reverts with `Nope()`. `Nope()` is a four-byte custom error selector, not the ABI-encoded `Error(string)` payload the hash compares against, so control reaches the final `revert UnexpectedRevert`. The test nevertheless expects `UseOfInvalidContract` and encodes three arguments where `UnexpectedRevert` carries two.

```
vm.expectRevert(  
    abi.encodeWithSelector(  
        AtomicQueueDerailBeforeTransferHook.UseOfInvalidContract.selector,  
        alice,  
        address(revertingQueue),  
        abi.encodeWithSelector(RevertingQueue.Nope.selector)  
    )  
);
```

The mismatch is in the test, not the hook: routing an arbitrary third-party revert to `UseOfInvalidContract` would misreport an unrelated failure as contact with the vulnerable queue, so the implementation's classification is correct. Beyond the failing assertion, `UseOfInvalidContract` appears in only this one expectation, attached to a stub that can never produce the reentrancy payload, so the branch that actually blocks a live `AtomicQueue.solve` has no passing assertion on its revert shape anywhere in the suite.



IMPACT

The suite delivered with the mitigation is red out of the box: 9 of 10 native tests pass and `testUseOfInvalidContractBlocksTransfers` fails with `UnexpectedRevert(0x7E5F4552091A69125d5DfCb7b8C2659029395Bdf, 0x2de7f6df)` against the expected `UseOfInvalidContract(0x7E5F4552091A69125d5DfCb7b8C2659029395Bdf, 0xA4AD4f68d0b91CFD19687c881e50f3A00242828c, 0x2de7f6df)`. Any CI gate running this suite blocks the emergency mitigation it is meant to ship.

Reviewers reading the suite are also told, incorrectly, that a queue reverting with any custom error is reported as `UseOfInvalidContract`, which misstates the error contract integrators would key on.

There is no asset loss and no on-chain state corruption; the demonstrated effect is the suite result and the missing coverage of the branch the mitigation exists to exercise.

RECOMMENDATION

Change `testUseOfInvalidContractBlocksTransfers` to expect `UnexpectedRevert(alice, abi.encodeWithSelector(RevertingQueue.Nope.selector))`, which is what the implementation correctly returns for an unrelated revert.

Then add a separate test that points the hook at a queue whose `solve` executes behind the solmate `nonReentrant` guard and asserts `UseOfInvalidContract`, so the branch the mitigation depends on is covered. The hook itself needs no change.

RESOLUTION

Paxos Labs

Fixed. The test has been fixed and a new test has been added.

[I-03] Probe burns full 30k stipend on every transfer

Info

Fixed

SUMMARY

`beforeTransfer` probes `atomicQueue.solve` via `staticcall` with a fixed `DERAIL_GAS_STIPEND = 30_000`. Outside a solve, `nonReentrant` passes its check and executes `locked = 2`; an SSTORE inside a static frame triggers an EVM exceptional halt.

An exceptional halt consumes every unit of gas forwarded to that frame, unlike `REVERT`, which returns unused gas to the caller. So every normal transfer pays the full stipend even though only about 3,200 gas of work happens before the halt.

Measured against the unchanged hook and queue:

```
probe gas used with 30k stipend: 30909
plain warm transfer:           3910
hooked warm transfer:         36393
hook overhead:                 32483
```

Forwarded amount, not work performed, sets the cost: raising the stipend to 100k burns 100,903 gas. With 30k, each hooked `transfer` or `transferFrom` pays about 32,500 extra gas, roughly nine times the



warm execution cost of the transfer itself. Inside a real solve, the guard reverts through `REVERT` after about 2,418 gas, so detection needs far less than 30,000.

Two smaller per-transfer costs compound this. `atomicQueue` is a storage variable read on each call (cold SLOAD, 2,100 gas) although it never changes after construction. The probe payload is rebuilt every call with `new address[]()`, `new bytes()`, and `abi.encodeCall`, although it depends only on the selector and empty arguments.

IMPACT

Every share transfer of every vault carrying this hook pays unnecessary gas overhead.

Integrations that forward a bounded amount of gas to `transfer` also face a hidden 30k gas floor. No funds move incorrectly.

RECOMMENDATION

Lower `DERAIL_GAS_STIPEND` to a value with clear headroom over both probe paths. The detection path needs about 2,418 gas and the halt path about 3,188 before halting, so `8_000` leaves roughly 2.5x margin:

```
uint256 internal constant DERAIl_GAS_STIPEND = 8_000;
```

Validated against the same queue and vault: with an 8k stipend, a drain attempt still reverts with `TRANSFER_FROM_FAILED`, plain transfers still pass, and hooked warm transfer drops from 36,393 to 14,293 gas, a 22,100 gas saving per transfer.

Measure on each target chain before deployment, since gas schedules differ. Also mark `atomicQueue` as `immutable`, and build the probe payload once rather than on every call. Add a comment noting that the normal path ends in an exceptional halt so future maintainers keep the stipend minimal.

RESOLUTION

Paxos Labs

Fixed. We have set the gas stipend to 8000.

[I-04] Hook installation leaves existing share-offer requests appearing solvable

Info

Acknowledged

LOCATION

AtomicQueueDerailBeforeTransferHook:beforeTransfer:52-64

SUMMARY

`viewSolveMetadata()` and `isAtomicRequestValid()` are meant to identify requests that `solve()` can actually execute: a request reported with no validation flags should be transferable through the path `solve()` uses.

Once the derail hook is installed on the BoringVault, any `solve()` offering BoringVault shares reaches the hook during `safeTransferFrom()` and reverts, because the queue is already inside `solve()`. The metadata functions do not account for this transfer restriction, so they keep reporting the request as valid (`flags == 0`) even though execution is impossible.



IMPACT

Existing AtomicQueue requests offering BoringVault shares become unexecutable while metadata still advertises them without an error flag.

Solvers may repeatedly submit transactions that deterministically revert. Because a single blocked legacy request aborts the whole solve batch, one such request can invalidate an otherwise solvable multi-user batch, causing persistent operational failure until the request is canceled or updated.

EXPLOIT PATH

1. A user calls `updateAtomicRequest(BoringVault, want, request)` with a nonzero share amount and approves AtomicQueue to spend the shares.
2. The vault installs `AtomicQueueDerailBeforeTransferHook`, making BoringVault share transfers through `AtomicQueue.solve()` fail.
3. A solver calls `viewSolveMetaData()` and receives `flags == 0`, since balance, allowance, deadline, and amount all remain valid.
4. The solver calls `solve()`; `safeTransferFrom()` invokes the hook, which detects the active queue lock and reverts, aborting the entire solve batch.

PREREQUISITES

An active AtomicQueue request uses the hooked BoringVault as its offer token, the vault's derail hook is installed, and the request otherwise has valid balance, allowance, deadline, and a nonzero amount.

RECOMMENDATION

Expose the transfer-blocked state so metadata rejects requests whose offer token cannot currently be transferred through the queue. For example, add a hook-aware validation path:

```
function isAtomicRequestExecutable(ERC20 offer) internal view returns (bool) {
    if (address(offer) == address(boringVault)) {
        return address(boringVault.hook()) == address(0);
    }
    return true;
}
```

Alternatively, when installing the derail hook, explicitly cancel or migrate all outstanding BoringVault-share requests and provide a user-facing cancellation function so they cannot remain advertised as solvable:

```
function cancelAtomicRequest(ERC20 offer, ERC20 want) external {
    delete userAtomicRequest[msg.sender][offer][want];
}
```

RESOLUTION

Paxos Labs

Acknowledged. The scope of this fix is the interaction between latent vault approvals made to the `AtomicQueue` and the vulnerable `solve` function. We consider the `AtomicQueue` to be deprecated and no longer support even legitimate transactions through it.



[I-05] Queue-wide reentrancy lock blocks unrelated BoringVault transfers

Info

Acknowledged

LOCATION

AtomicQueueDerailBeforeTransferHook:beforeTransfer:52-61

SUMMARY

The transfer hook is meant to block only BoringVault share transfers that flow through the vulnerable AtomicQueue settlement, while leaving unrelated BoringVault transfers and unrelated queue operations untouched.

Instead, the hook probes only the configured queue's global `nonReentrant` state. Any active `solve()` call holds that lock, including solves whose assets have nothing to do with BoringVault shares.

Because `solve()` invokes an arbitrary solver callback before releasing the lock, a callback that performs a normal `BoringVault.transfer` causes the hook to observe the `REENTRANCY` marker and revert. The set of valid share transfers therefore depends on unrelated queue activity, with no share-accounting change required for the failure to manifest.

The unused `from` parameter carries no authorization or path information, so the hook cannot distinguish a queue-mediated transfer from an ordinary one and blocks unconditionally while any solve is active.

IMPACT

A solver or composed callback can make otherwise valid BoringVault transfers revert for the duration of an AtomicQueue operation, even when neither asset in the request is BoringVault shares. This is a liveness and composability failure with no fund loss.

Since the block is permissionless, an unrelated solve can synchronously censor an otherwise authorized share movement, and the failure repeats for as long as the queue is busy.

The damage crosses assets and vaults: when two vault hooks share the same AtomicQueue, a solve involving Vault A can block a transfer of Vault B shares. Composable flows reached through the `finishSolve` callback, such as flash-loan flows and multi-hop arbitrage or rebalance/collateral operations, become non-composable while any solve runs.

EXPLOIT PATH

1. A solver starts `AtomicQueue.solve` for TokenA and TokenB, neither of which is the BoringVault share.
2. The queue invokes the solver's `finishSolve` callback while its `nonReentrant` lock is still held.
3. The callback calls `BoringVault.transfer(recipient, amount)` for an unrelated share movement.
4. The hook observes the queue's `REENTRANCY` state and rejects the transfer, aborting the callback and the whole solve transaction.

PREREQUISITES

The hook must be installed, and a BoringVault transfer must be attempted from within the AtomicQueue's external solver callback (or any call tree executed while a `nonReentrant` queue function is active). No special role is required; the triggering solve is permissionless.

RECOMMENDATION

A queue-side flag such as `bool public protectedShareTransfer` cannot be used here, because the vulnerable AtomicQueue is already deployed and immutable, so no new state can be added to it.



This is the same over-blocking root cause as the rest of this cluster and is liveness-only (atomic revert, no fund loss). Two safe options:

1. Accept it for the temporary freeze period. It is strictly less restrictive than the blanket-freeze hook it replaces.
2. If precise blocking is required, adopt the richer interface the newer BoringVault exposes, `beforeTransfer(from, to, msgSender, amount)`, where the vault forwards the true initiator and amount. Block additively only on a detected queue-driven share drain and keep the default allow for ordinary transfers.

Do not gate the block on `msg.sender == address(atomicQueue)`: the hook's caller is always the vault, so that guard never fires and the hook fails open.

RESOLUTION

Paxos Labs

Acknowledged. The scope of this fix is the interaction between latent vault approvals made to the `AtomicQueue` and the vulnerable `solve` function. We consider the `AtomicQueue` to be deprecated and no longer support even legitimate transactions through it.

[Self-report I-01] Caller can underfund reentrancy probe gas stipend, bypassing it

InfoFixed

LOCATION

`AtomicQueueDerailBeforeTransferHook:beforeTransfer`

SUMMARY

The `beforeTransfer` hook probes the `AtomicQueue` with `staticcall{ gas: DERAILED_GAS_STIPEND }` and treats empty revert data as "not reentrant". However, a probe that runs out of gas partway through its `REENTRANCY` revert also returns empty data. Under EIP-150, the probe receives at 63 /64 of the available gas, so a caller can limit its gas while retaining some which may be used to continue execution of the main transaction. In theory, an attacker could tune the gas limit of a transaction to bypass the hook.

IMPACT

Under certain circumstances, an attacker could run malicious `solve()` calls against users' latent vault share approvals to the `AtomicQueue`, and drain those shares as if the hook were not installed.

Currently, those circumstances are not possible. Underfunding the probe requires entering the hook with less than about 4.3k gas. The vault is then left with only about 65 gas to finish the transfer. The transfer reverts, and `solve()` reverts with it.

The viability of this attack is dependent on specific EVM gas costs which may change in a future upgrade.



REMEDIATION

Commit [09990c9](#) adds a gas check before the probe, as defense in depth:

```
uint256 gasAvailable = gasleft();
if (gasAvailable - gasAvailable / 64 < DERAIl_GAS_STIPEND) {
    revert InsufficientGasForProbe(
        gasAvailable, DERAIl_GAS_STIPEND
    );
}
```

This guarantees that the probe always receives the full stipend which we have defined at the contract level.

We acknowledge that the same theoretical EVM upgrades we are protecting against could invalidate the stipend itself. However, we believe that a stipend of 8000 gas (suggested in https://github.com/PashovAuditGroup/PaxosLabs_September26_MERGED/issues/3) provides a reasonable margin of safety and we expect that any such upgrade would be publicised ahead of time, allowing us to deploy a new hook with a higher stipend.